Authentication, OAuth, and App Check

Firebase Auth providers, the OAuth handoff, and App Check enforcement

Dinu-Ștefan Rusu ADMD

What you should leave with

Authentication vs authorization

Who the user is versus what they may do, and which layer answers each.

Firebase Auth in Dart

Sign-up, sign-in, reset, verification, sign-out, and authStateChanges.

OAuth and tokens

The client-side credential handoff, and what an ID token proves.

Rules and App Check

Rules that bind a request to a uid, and enforcement that keeps scripts out.

1 Authentication fundamentals

Contents Page

1 **Authentication fundamentals** 3
2 Firebase Auth in Dart 7
3 OAuth and identity tokens 17
4 Data protection and App Check 23

Authentication and authorization

“Is this user really who they claim to be?”

Authentication checks a credential: a password, a provider token, a passkey. It issues a session.

“Now that I know who this is, what may they touch?”

Authorization decides, per request, which documents this identity may read, create, update, or delete.

Firebase Auth only answers the first question. The second is answered by your security rules and by your own backend. A signed-in user is not an authorized user.

Authentication factors and MFA

Factor	What it means	Examples	How it fails
Knowledge	something you know	password, PIN, security question	phished or reused
Possession	something you have	phone code, passkey, hardware key	SIM-swappable, unlike a passkey
Inherence	something you are	fingerprint, Face ID, iris	never leaves the device

Multi-factor authentication (MFA) needs factors from two different rows. A password plus a security question is one factor, asked twice. Firebase supports SMS and a time-based one-time password (TOTP) as a second factor.

Managed authentication

What Firebase gives you

- Password hashing, salting, and rotation you never touch
- Verification and reset mail, delivered and rate-limited
- Twenty-plus federated providers behind one Dart API
- Token minting, refresh, and enumeration protection, on by default

What you hand over

- Identity records live in a Google project, not your database
- Pricing tracks monthly active users past the free tier
- Their email templates and flows: customizable, not arbitrary
- The uid becomes your primary key; migrating it is real work

2

Firebase Auth in Dart

Sign-up, sign-in, account lifecycle

Contents

		Page
1	Authentication fundamentals	3
2	**Firebase Auth in Dart**	7
3	OAuth and identity tokens	17
4	Data protection and App Check	23

Auth providers and packages

Category	Provider	Packages	Worth knowing
Native	Email, password, phone	firebase_auth	enumeration protection is on by default
Federated	Google, Apple, others	firebase_auth plus a provider package	Google is the flow this lecture implements
Anonymous	Anonymous	firebase_auth	a real uid before sign-up; link it later

One user, one uid, however many providers. Firebase keys the account on the email address by default, so signing in with Google using that same address lands on the same user.

Creating an account

```
final auth = FirebaseAuth.instance;
Future<void> signUp(
    String email, String pass) async {
  try {
    final cred = await auth
        .createUserWithEmailAndPassword(
    email: email.trim(), password: pass);
    await cred.user!
        .sendEmailVerification();
  } on FirebaseAuthException catch (e) {
    throw AuthFailure(
        mapSignUpError(e.code));
  }}
```

Three things to notice

The call also signs the user in immediately. The password never reaches your code again.

Send the verification mail here, not later. Keep unverified accounts out of your data.

Sign-up still reveals existence: email-already-in-use is unavoidable. Rate limiting and App Check help.

Signing in and error messages

```
Future<void> signIn(
    String email, String pass) async {
  try {
    await auth.signInWithEmailAndPassword(
        email: email.trim(),
        password: pass);
  } on FirebaseAuthException catch (e) {
    throw AuthFailure(
        mapSignInError(e.code));
  }
}
```

Why one message

Since late 2023, Firebase no longer says which half was wrong: an unknown email and a wrong password both arrive as invalid-credential.

Only two codes get their own text: too many attempts, and offline. Neither one reveals whether the account exists.

Never give user-disabled its own message either: that confirms the account exists.

Account enumeration through the login form

A message like “no user found for that email” turns the login form into a lookup service. Sending it ten thousand addresses reports which ones have accounts, with no password and no breach involved.

That resulting list is itself sensitive data. On a dating app, a health app, or a debt-management app, membership alone is the sensitive fact. It is also the first step of a credential-stuffing run: confirm which accounts exist, then attack only those addresses.

Email enumeration protection

Situation	Older error code	Current behavior
Unknown email	user-not-found	invalid-credential
Wrong password	wrong-password	invalid-credential
Reset for an unknown address	user-not-found	succeeds silently
fetchSignInMethodsForEmail	the list of providers	an empty list; the method is deprecated

Identical message, identical behavior, whatever went wrong. Back it with rate limiting and App Check, and confirm the protection is still on, in Console under Authentication then Settings.

authStateChanges

```
// Don't check-and-push. Listen, and let the UI follow.
StreamBuilder<User?>(
  stream: FirebaseAuth.instance.authStateChanges(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const SplashScreen();  // session still restoring
    }
    final user = snapshot.data;
    if (user == null) return const LoginScreen();
    if (!user.emailVerified) return const VerifyEmailScreen();
    return const HomeScreen();
  },
)
```

Auth state streams

1. **authStateChanges()** fires on sign-in, sign-out, and once at startup when the session is restored from disk.
2. **idTokenChanges()** fires on all of that, plus every hourly token refresh and every custom-claim change.
3. **userChanges()** fires on all of that, plus profile edits such as a changed display name.

Never cache currentUser in a field of your own. It goes stale, and stale auth state is a security bug. Route guards belong to the router: feed this stream into go_router's refreshListenable (Lecture 11).

Reset, verification, and sign-out

```dart
// Password reset: never reveal whether the account exists.
await auth.sendPasswordResetEmail(email: email.trim());

await auth.currentUser?.sendEmailVerification();
await auth.currentUser?.reload();
final verified = auth.currentUser?.emailVerified ?? false;

await auth.signOut();
await GoogleSignIn.instance.signOut();
```

Enforce email verification. Enforce it in the auth gate and again in your security rules, with request.auth.token.email_verified.

Linking, re-authentication, second factors

Account linking

linkWithCredential attaches Google to an existing email account, or upgrades an anonymous user without losing data.

Re-authentication

Changing an email or password needs a recent login. Catch requires-recent-login and reauthenticate.

Second factors

SMS and TOTP enrollment via multiFactor. local_auth's Face ID is a device lock, not a factor.

```
// Anonymous to real account: same uid, same data.
final cred = GoogleAuthProvider.credential(idToken: idToken);
await FirebaseAuth.instance.currentUser!
    .linkWithCredential(cred);
```

3

OAuth and identity tokens

Credential handoff, ID tokens

Contents Page

1 Authentication fundamentals 3
2 Firebase Auth in Dart 7
3 OAuth and identity tokens 17
4 Data protection and App Check 23

Client-side credential handoff

1. **The app asks Google to sign the user in.** `GoogleSignIn.instance.authenticate()`
2. **Google returns an OAuth ID token.** a JSON Web Token (JWT) signed by Google, naming this user
3. **The app wraps it as an AuthCredential.** `GoogleAuthProvider.credential(...)`
4. **Firebase verifies it and opens a session.** `signInWithCredential(cred)`
5. **Your own backend verifies the Firebase token.** `admin.auth().verifyIdToken(idToken)`

Every step above runs on the device. Firebase never sees the Google password: it checks Google's signature, then mints its own session.

Google sign-in flow

```dart
import 'package:google_sign_in/google_sign_in.dart';
// google_sign_in 7.x: initialize once, at app start.
await GoogleSignIn.instance.initialize();

Future<UserCredential> signInWithGoogle() async {
  final account = await GoogleSignIn.instance.authenticate();
  final idToken = account.authentication.idToken;
  final credential =
      GoogleAuthProvider.credential(idToken: idToken);
  return FirebaseAuth.instance.signInWithCredential(credential);
}
```

Social sign-in setup

- Android release builds need the SHA-1 and SHA-256 fingerprints registered in the Firebase console, or sign-in fails silently.
- A canceled account picker throws a GoogleSignInException with code `canceled`. That is a user action, not an error to report.
- On Apple platforms: `sign_in_with_apple`, then `OAuthProvider('apple.com')` with the ID token and the raw nonce you generated.
- Offer Sign in with Apple on iOS whenever you offer any other social login. App Review enforces it.

ID token vs refresh token

	ID token	Refresh token
Lifetime	one hour, auto-refreshed	until revoked, or password change
Format	signed JWT: uid, email, claims	opaque string, unreadable
Storage	memory, via getIdToken()	Keychain or Keystore
Sent to API	yes, as a header	never; stays on the device

A JWT is signed, not encrypted. Anyone holding it can read every claim, so put nothing secret in a custom claim.

Backend token verification

```
final token = await FirebaseAuth.instance.currentUser!.getIdToken();
await http.post(url,
    headers: {'Authorization': 'Bearer $token'},
    body: jsonEncode({'item': itemId}));
```

```
const token = req.get('Authorization').split(' ')[1];
const decoded = await admin.auth().verifyIdToken(token, true);
const uid = decoded.uid;
await db.collection('orders').add({ uid, item: req.body.item });
```

Anyone can claim to be someone else in a request body. The uid you act on has to come out of a token you verified, and from nowhere else.

4 Data protection and App Check

User profiles, security rules, App Check

Contents

		Page
1	Authentication fundamentals	3
2	Firebase Auth in Dart	7
3	OAuth and identity tokens	17
4	**Data protection and App Check**	23

Password storage

The identity provider holds the credential. Firebase keeps a salted hash on Google's infrastructure; once account creation returns, the plaintext password is gone from your process.

Your database should store an identifier and profile data, and nothing that could authenticate anyone: no password, no PIN, no security answer, no copy of a token.

The Firebase User object is for identity: uid, email, displayName, photoURL.
Application data goes in your own collection, keyed by the uid.

User profile documents

```
// After sign-up: a profile document keyed by the uid.
final uid = cred.user!.uid;
await FirebaseFirestore.instance
    .doc('users/$uid')
    .set({
  'displayName': name,
  'createdAt': FieldValue.serverTimestamp(),
});
// No password. No token. No PIN.
```

The uid is the join key. It connects the identity system to your data, and that join is what the next two slides defend.

Uid-based security rules

A rule for a per-user profile document, split by verb.

```
match /users/{userId} {
  function isOwner() {
    return request.auth != null && request.auth.uid == userId;
  }
  allow read:   if isOwner();
  allow create: if isOwner()
                && !request.resource.data.keys().hasAny(['role']);
  allow update: if isOwner()
                && request.resource.data.diff(resource.data)
                     .affectedKeys().hasOnly(['displayName']);
  allow delete: if false;   // deletion runs on the backend
}
```

Insecure rule example

A rule that does not bind

```
allow create: if request.auth != null;
```

- Any signed-in user, even one created seconds ago, can write a document at someone else's uid and take over their profile.
- `request.auth != null` is authentication. Authorization means binding the request to the exact path it touches.
- Split the verbs: create, update, and delete need different conditions. A blanket write hides all three.

Rules run on Google's servers, so a client cannot bypass them.

App Check overview

Your Firebase configuration ships inside the binary: an API key, a project ID, an app ID. It identifies the project. It was never a secret.

An attacker can unzip the Android package (APK), read that configuration, and call the Firestore or Storage REST API directly from a script. None of the limits your UI imposes apply there.

Authentication answers who. App Check answers where from: is this request coming from a genuine build of this app, on a genuine device?

App Check, Auth, and security rules

App Check

Is this a genuine build of my app? Protects the developer: abuse and billing fraud.

Firebase Auth

Is this a known, valid user? Protects the user: unauthorized access.

Security rules

May this user do this, here? Protects the data: misuse by a valid user.

Enabling App Check

```
import 'package:firebase_app_check/firebase_app_check.dart';
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
      options: DefaultFirebaseOptions.currentPlatform);
  await FirebaseAppCheck.instance.activate(
    androidProvider: kDebugMode
        ? AndroidProvider.debug : AndroidProvider.playIntegrity,
    appleProvider: kDebugMode
        ? AppleProvider.debug : AppleProvider.appAttest,
  );
  runApp(const App());
}
```

Choosing an attestation provider

Platform	Provider	Note
Android	Play Integrity	confirms the app was installed by Play, unmodified
iOS 14 and later	App Attest	a per-install hardware-backed key
Older iOS	DeviceCheck	device-level attestation, weaker but broadly available
Web	reCAPTCHA Enterprise	the browser equivalent of the same check
Dev, CI	Debug provider	emulators and simulators cannot attest for real

Enforcement and debug tokens

1. **Ship a build that calls activate().** Clients start attaching App Check tokens to every request.
2. **Watch the metrics.** Console, under App Check: verified traffic versus unverified.
3. **Wait for old versions to drain.** Unverified traffic has to fall to near zero first.
4. **Press Enforce, per product.** Firestore, Storage, Cloud Functions, Authentication, Realtime Database.

Until you press Enforce, App Check only measures. A debug token is a permanent bypass of everything you just enabled: never commit one, never paste one in chat.

Summary

1. **Authentication answers who; authorization answers what.** Firebase Auth and your security rules are different layers.
2. **One generic message for every failed sign-in.** Never confirm, in the UI, that an account exists.
3. **Verify the ID token on your backend.** A uid supplied by the client proves nothing on its own.

Further reading

Firebase Auth for Flutter · Rules and Auth · App Check docs · OWASP MASVS

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel